



华侨大学

HUAQIAO UNIVERSITY

本科毕业论文（设计）

题目：基于大模型的智能对话系统

设计与实现

学 院	计算机科学与技术学院
专 业	软件工程
年 级	2021 级
学 号	2125121024
姓 名	黄楊
指导老师	蔡奕侨

华侨大学教务处印制

2024 年 6 月

摘要

人工智能技术持续发展，大模型于自然语言处理领域呈现出出色的语言理解以及生成能力，成为推动智能对话系统前行的关键动力，不过当下主流对话系统在模型选择跟上下文自定义等方面依旧存在不少限制，本文依托大模型技术，设计并实现了一种灵活且可扩展的智能对话系统，用以契合不同用户在多种场景下的需求。

本系统运用 Go 语言开展后端开发，借助 Gin 框架搭建 Web 服务，前端运用 Vue3 框架实现交互界面，数据库采用 MySQL 以及火山云 OSS 进行存储，同时整合 VikingDB 向量数据库用于向量检索，在技术架构层面，系统融合了字节跳动开源的 Eino 框架，支持多家主流大模型 API 的接入与调用。本课题先是对智能对话系统的需求做了详细分析，囊括环境需求、技术需求以及功能需求，接着进行了全面的架构设计以及数据库表设计，依据设计完成各个模块的开发实现，包含登录模块、会话模块、知识库模块以及模型配置模块，最后完成系统测试与部署。

经过系统的实现以及功能测试验证，本课题设计并实现的智能对话系统拥有良好的可扩展性与灵活性。依靠灵活挑选指定模型、灵活配置上下文以及构建个人知识库，可较好地契合用户在特定场景下的个性化需求，本课题的研究有一定的理论与实践价值，为后续智能对话系统的设计与优化提供了可行的路径与参考依据。

关键词：智能对话系统；大模型；Go 语言；知识库

ABSTRACT

With the continuous development of artificial intelligence, large-scale models have demonstrated significant language understanding and generation capabilities in the field of natural language processing. These large-scale models have become the key force driving the advancement of intelligent dialogue systems. However, the current mainstream dialogue systems have problems in terms of mode selection and context customization. There are still many challenges. This paper, with the aid of large model technology, proposes and develops a flexible and scalable intelligent dialogue system, which can meet the various needs of different users in different scenarios.

The back end of the system is built with the Go language, and the Gin framework is also used to establish Web services. The front end uses the ve3 framework to create an interactive interface. Data storage is managed by MySQL and Volcano Cloud OSS, and the VikingDB vector database is also integrated, thus enabling efficient vector retrieval. In terms of technical architecture, the system adopts the Eino framework open-source by ByteDance, achieving seamless integration and invocation of multiple leading large-scale apis. At the very beginning of this project, a comprehensive analysis was conducted on the requirements of the intelligent dialogue system, which included environmental aspects, technical aspects, and functional aspects. Then, detailed architectural design and database schema planning were carried out. Each module, such as the login module, session management module, knowledge base module, and model configuration module, was developed and implemented in accordance with the design specifications. The system underwent comprehensive testing and was then deployed.

Through the development and functional evaluation of the system, the intelligent dialogue system proposed in this project has demonstrated excellent scalability and adaptability. It enables users to select from multiple large models, customize the context, and build personalized knowledge bases, thus successfully addressing specific scenario-based user needs. The research of this project has crucial theoretical and

ABSTRACT

practical value, and can provide feasible methods and a referable framework for the design and optimization of future intelligent dialogue systems.

Keywords: Intelligent Dialogue System; Large Language Model; Go Language; Knowledge Base

目录

第一章 绪论.....	1
1.1 选题背景和意义	1
1.2 国内外研究现状	2
1.3 主要研究内容	2
1.4 本文篇章结构	3
1.5 本章小结	4
第二章 智能对话系统的需求分析	5
2.1 系统环境需求	5
2.2 系统技术需求	5
2.2.1 语言和框架技术	5
2.2.2 数据存储技术	6
2.2.3 大模型技术	6
2.3 系统功能性需求	7
2.3.1 功能性需求概述	7
2.3.2 登录模块的功能需求	8
2.3.3 会话模块的功能需求	8
2.3.4 知识库模块的功能需求	10
2.3.5 模型配置模块的功能需求	11
2.4 系统非功能性需求	11
2.5 本章小结	12
第三章 智能对话系统的概要设计	13
3.1 系统的架构设计	13
3.2 系统的模块设计	14
3.3 数据库设计	15
3.3.1 数据库 ER 图	15
3.3.2 数据库表设计	16
3.4 本章小结	18
第四章 智能对话系统的详细设计	19
4.1 登录模块	19
4.1.1 OAuth2.0 登录	19

4.1.2 JWT 登录验证和状态保持	20
4.2 会话模块.....	21
4.2.1 对话流程图	21
4.2.2 删除会话流程图	23
4.3 知识库模块.....	24
4.3.1 上传文件.....	24
4.3.2 下载文件.....	25
4.4 模型配置模块.....	25
4.5 本章小结.....	26
第五章 智能对话系统的实现和测试.....	28
5.1 登录模块的实现	28
5.2 对话模块的实现	28
5.2.1 会话操作	28
5.2.2 修改会话配置	30
5.2.3 对话功能.....	30
5.3 知识库模块的实现	32
5.4 模型配置模块的实现.....	33
5.5 系统测试.....	35
5.5.1 系统测试环境	35
5.5.2 系统功能测试	36
5.6 本章小结.....	38
第六章 总结和展望.....	39
6.1 总结	39
6.2 展望	39
参考文献	41
致谢	42

第一章 绪论

1.1 选题背景和意义

人工智能技术快速发展，在自然语言处理领域取得了显著的进步，特别是大模型^[1]的研究和应用受到了广泛关注。大模型的应用在很大程度上是数据应用^[2]，通过对海量文本数据的训练，能够生成语义连贯、符合上下文逻辑的自然语言文本，广泛应用于多个领域。对话系统作为大模型的重要应用方向之一，吸引了大量研究者和开发者的关注。例如，OpenAI 的 ChatGPT、阿里的通义千问、百度的文心一言以及字节跳动的豆包等对话系统，已经在智能客服、教育、内容创作等场景中展现了强大的潜力。

大模型驱动的对话系统在诸多方面实现了突破性发展，然而，当前的应用依旧面临一些急需解决的问题，其一当下主流应用里可使用的大模型种类相对单一，大多时候绑定自家大模型，用户若想使用多个大模型，就只能在多个应用之间来回切换，其二现有的对话系统在上下文自定义能力方面存在欠缺，难以很好地契合用户在特定场景下的个性化需求。

针对上述这些问题，本研究的选题是凭借大模型的技术优势，融合 RAG 检索增强技术，探寻智能对话系统的设计与实现方案，此系统采用更为开放的技术架构，可支持集成多家大模型，让用户可以依据需求挑选适宜的模型，并且支持用户自定义检索知识库，便于用户引入专属领域的相关知识，提高系统的实用性。在上下文管理方面，系统会提供高度可配置性的上下文范围调整功能，使用户可更精准地控制对话语境，实现更高质量的对话生成。

本系统的目标用户群体主要是有大模型基础知识的开发者和技术人员，凭借提供高度自定义化的功能设计，可提升用户体验，降低使用成本，同时为智能对话系统的研究与实践开拓新的方向。借助这个平台，用户可以在多个模型之间灵活切换，灵活管理上下文，为推动智能对话技术的应用落地提供有力支撑。

1.2 国内外研究现状

OpenAI 公司于 2020 年推出 GPT-3 后，促使 AI 技术在不同领域广泛被应用。GPT-3 以及后续版本如 GPT-4^[3]，具备强大的自然语言处理能力，使科研人员能够高效地撰写学术论文、进行复杂数据的分析并自动生成研究成果^[4]。这些语言模型能理解和生成多种语言文本，极大提升机器与人类互动效率，在翻译、问答系统和内容创作等方面呈现巨大潜力。GPT 系列模型不光在文本生成方面表现优异，在图像描述、情感分析、语义理解等领域也取得令人赞叹的成绩，使 AI 在法律、教育、医疗、金融、娱乐等多个行业得到更广泛应用，这些行业引入 AI 技术后工作效率和服务质量得以提高。

随着大模型的井喷式发展，推理成本、API 成本的下降是整个行业急迫需要的，各大互联网公司开启了大模型“价格战”^[5]。过去训练和运行大型 AI 模型成本一般很高，只有少数顶级公司和科研机构能承担，不过随着计算资源逐渐普及以及算法优化有进展，推理成本和 API 调用费用慢慢下降，促使更多中小企业和开发者能进入这个市场，成本降低让更多公司能接入并使用 AI 模型，还推动了 AI 服务普及和创新，使 AI 技术能应用到更广泛场景。

近期，国内新兴 AI 企业 DeepSeek 研发推出 DeepSeek-V3 模型，凭借其出色性能和低廉训练成本，在业界引起震动，和以往大型 AI 模型相比，DeepSeek-V3 采用更高效训练方法和优化技术，成功大幅降低训练成本，这让更多中小型企业在不超预算情况下，能用深度学习技术解决实际问题，推动 AI 技术在更多行业落地应用。DeepSeek 这一技术突破改变了 AI 市场竞争格局，也为行业发展提供新思路 and 可能性，随着 DeepSeek-V3 模型推出，预计会有更多创新性应用和解决方案出现，推动全球 AI 技术进步与普及。

1.3 主要研究内容

本课题将遵循传统的瀑布模型^[6]研究方法，确保研究的流程规范、有序，并根据软件工程中面向对象的方法，对基于大模型的智能对话系统进行设计和实现。使用 Go^[7]高级语言在开发工具 Goland 中进行代码实现，采用前后端^[8]分离的

开发方式完成对系统的开发，最后进行系统部署和上线。以下是本课题的主要研究内容：

1. 研究开发智能对话系统所需要使用到的相关技术，特别是 Go 语言开发和大模型开发的相关技术。前端和后端分别使用 Vue3 和 Gin 框架，数据处理使用 Gorm 框架，数据存储使用 MySQL 和火山云的对象存储服务。

2. 完成智能对话系统的需求分析。在系统开发初期，深入分析用户交互场景和用户需求，明确系统需支持的核心功能，包括登录、多轮对话管理、对话配置和知识库管理等。并结合实际使用场景，明确系统的非功能性需求。

3. 完成智能对话系统的设计。在明确系统需求的基础上，完成对系统的架构设计，功能结构设计、数据模型设计、前后端通信接口设计。同时绘制系统流程图、数据结构图与数据库 E-R 图，辅助开发实现。

4. 完成智能对话系统的实现、测试和部署。按照设计方案进行系统编码实现，实现完成后，通过功能测试和接口测试，确保系统的稳定性与可靠性。最终部署到腾讯云服务器，保证系统上线后的可用性与可持续运行能力。

1.4 本文篇章结构

本文分为 6 个章节进行论述，组织结构如下：

第一章：分析大模型及对话系统发展现状，指出当前基于大模型的对话系统存在的不足，进一步探索了更具灵活性和开放性的智能对话系统形式，明确了本文的主要研究方向。

第二章：主要是智能对话系统需求分析。首先介绍了系统开发过程中使用到的相关技术，然后以用例图的方式对每个模块进行分析。

第三章：主要是智能对话系统的概要设计。首先展示了完整的部署架构和技术架构，再展示系统功能结构图，然后使用 ER 图展示了数据库表设计。

第四章：主要是智能对话系统的详细设计，从系统的业务流程和程序流程出发，绘制系统各个功能的交互时序图和程序流程图，并使用文字说明进行了介绍。

第五章：主要是对系统各模块进行了代码实现，截取了系统各个模块的功能页面作展示，并完成对系统的功能测试和测试报告。

第六章：对本次课题研究的内容以及系统的设计实现进行了总结，并对系统的后续版本作了展望。

1.5 本章小结

本章介绍了课题的研究背景与意义，分析了国内外大模型及智能对话系统的发展现状，明确了本研究的主要内容和技術路线，为后续系统设计与实现打下了理论基础。

第二章 智能对话系统的需求分析

2.1 系统环境需求

为了保障智能对话系统在开发、测试和部署各阶段的顺利进行，需要对系统环境进行合理规划与配置。系统环境主要分为开发环境、测试环境和部署环境，具体如下：

开发环境：本地开发环境基于 Ubuntu 22.04 操作系统，具有良好的稳定性与兼容性。后端采用 Go 语言作为主要开发语言，集成开发工具使用 JetBrains GoLand。GoLand 是专为 Go 语言开发推出的一款集成开发环境，提供了完整的开发、调试、测试能力，并拥有代码自动提示功能，提供各种部署工具帮助开发人员整合资源，简化了开发流程，极大地提升了整体开发效率。使用 Git 作为版本控制，GitHub 作为远程仓库存储代码。

测试环境：为保障系统功能的完整性与稳定性，搭建本地测试环境用于单元测试、接口测试。测试数据库独立部署，避免对实际数据造成干扰。借助 Postman 进行接口测试，借助 Go 自带测试工具 `testing` 包进行单元测试。

部署环境：系统部署采用腾讯云轻量应用服务器，为 Linux 系统，配置为 2 核 CPU、2GB 内存，符合小型智能对话应用的实际部署需求。部署过程采用 SSH 与云服务器通信发送指令，进行服务拉起与资源同步，方便后期维护与迭代升级。

2.2 系统技术需求

2.2.1 语言和框架技术

后端采用 Go 语言进行开发。Go 是 Google 开发的一种静态强类型、编译型、并发友好且具有垃圾回收机制的编程语言，因其出色的并发处理能力、编译速度快以及部署简便等特性，广泛应用于构建高性能服务端系统。Go 提供原生的协程与通道机制，在构建并发、分布式系统时具有天然优势，尤其适合实现响应速度要求较高的对话服务。

后端框架层面，系统采用 Gin 作为 Web 服务框架。Gin 是一款基于 Go 语言的轻量级 Web 框架，具有路由性能优越、启动速度快、API 设计清晰等特点。Gin 支持中间件机制、参数绑定、错误处理等功能，能够快速搭建稳定、易扩展的 Web 服务，是当前 Go 语言 Web 开发的主流选择之一。

数据库访问层使用 Gorm 框架。Gorm 是一款功能强大、易于使用的 ORM 框架，支持模型定义、自动迁移、事务管理、查询构建器等功能。通过 Gorm，开发者可以在 Go 语言中以面向对象的方式操作关系型数据库，提升开发效率，降低手动编写 SQL 的复杂性和出错率。

前端部分采用 Vue3 框架进行开发。Vue3 是一款现代化的渐进式 JavaScript 框架，具备响应式数据绑定、组件化开发、虚拟 DOM、组合式 API 等特性。

2.2.2 数据存储技术

MySQL 是一个开源的关系型数据库管理系统^[9]。MySQL 支持多种存储引擎，最常用的包括 MyISAM、InnoDB 等，也因它们各自不同的特性，使得 MySQL 可以满足绝大数的场景^[10]。因为 MySQL 是一个开源的数据库，考虑到成本问题，且 MySQL 满足我们的使用场景，所以本次数据库我们使用的是 MySQL 数据库。

知识库文件和对话过程的图像使用火山云的对象存储服务进行存储。因为对话过程的图片要通过 URL 传输给大模型，必须公网可以访问，因此在创建 OSS 的时候要设置 bucket 的策略为公有读。

VikingDB 是向量数据库，支持对嵌入向量的高效检索，用于 AI 语义相关性检索即 RAG 场景中使用。

2.2.3 大模型技术

大模型应用开发框架采用 Eino。Eino 是字节跳动公司今年开源的框架，基于明确的“组件”定义，提供强大的流程“编排”，覆盖开发全流程，旨在帮助开发者以最快的速度实现最有深度的大模型应用。它从开源社区中的诸多优秀 LLM 应用开发框架，如 LangChain 和 LlamaIndex 等获取灵感，同时借鉴前沿研究成

果与实际应用，提供了一个强调简洁性、可扩展性、可靠性与有效性，且更符合 Go 语言编程惯例的 LLM 应用开发框架，支持 Function Call^[11]。

2.3 系统功能性需求

2.3.1 功能性需求概述

需求分析最主要的是在业务上分析需要实现什么，而不应该过多的关注怎么实现，以确保业务场景完整的传递给系统设计人员和开发人员。本次系统需求分析选用其中具有代表性的工具用例图来进行分析，通过用例图我们可以快速了解业务场景。

如图 2.1 所示，系统角色可分为两类：用户和管理员。普通用户可以登录系统，使用对话功能，并可查看和管理个人的历史对话记录。此外，用户还可以访问知识库模块，检索已有内容或上传相关文件，以提升对话系统的回答的准确性与个性化。管理员除了拥有用户的全部权限外，还可以对系统内的大模型服务进行配置与管理，包括添加新模型、修改现有模型信息以及对模型进行激活或关闭操作，从而保障系统对接模型服务的灵活性和稳定性。

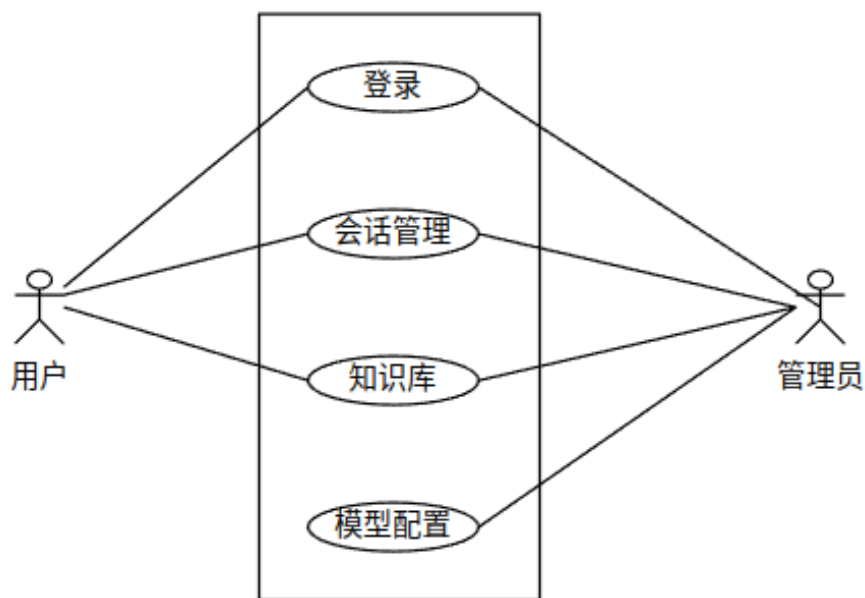


图 2.1 系统用例图

2.3.2 登录模块的功能需求

登录模块是系统用户身份识别和权限控制的入口，保障系统的安全性和可控性。

1. 管理员登录：管理员通过系统预设的账号密码进行登录，账户密码通过后端接口进行验证。管理员身份一旦验证通过，即可访问系统的高级管理功能，大模型配置功能。

2. 用户登录：普通用户仅提供 GitHub 第三方授权登录，用户点击“GitHub 登录”按钮后将跳转至 GitHub 授权页面。登录成功后，使用 Github 账户的用户名和头像作为本系统的用户名和头像。该信息将作为用户在本系统中的唯一身份标识。用户无需注册，只需一次授权，即可完成快速登录。

3. 身份鉴权：除了登录接口外，所有对外开放的接口在调用时都必须验证用户的身份信息。只有登录后的用户才能访问系统资源，该机制能够有效保障数据安全，防止未授权访问。

4. 登录状态保持：为了提升系统的易用性和用户体验，系统支持自动登录功能。即在用户成功登录后，在一段时间内（如一周）再次访问系统时，可免登录直接进入系统界面，无需重复身份验证。

2.3.3 会话模块的功能需求

如图 2.2 所示，会话模块是智能对话系统的核心功能之一，主要用于实现用户与大模型之间的交互。该模块支持两种对话。

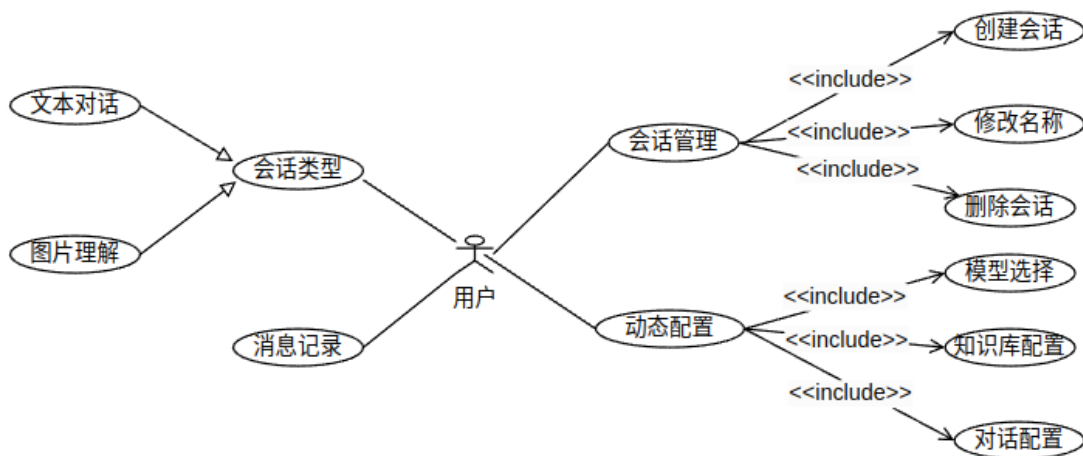


图 2.2 会话模块用例图

1. 会话管理功能：用户可自主创建新会话，修改会话名称，删除会话。
2. 会话类型：系统支持文本对话和图片理解两类对话形式，文本对话支持用户以自然语言进行提问与交流，图片理解则支持用户上传图片，由大模型对图像内容进行识别和分析，生成自然语言回应。
3. 消息记录：用户与模型之间的每轮对话消息都会进行记录，用户可查看历史消息。
4. 模型选择：每次对话都可以基于指定的模型。用户可选择通用大模型进行闲聊，或选择特定模型进行专业领域的问答。
5. 知识库配置：用户开启 RAG 功能后，可选择指定的一个或多个知识库进行检索，实现对特定领域知识的补充。
6. 对话配置：系统支持用户在对话过程中动态调整会话配置，实现灵活的上下文控制。用户在会话过程中修改配置项并保存，系统将在后续对话中即时生效，提升使用体验。主要配置项包括：
 - （1）最大 Token 数：限制大模型生成文本的最大长度，防止响应过长引起上下文偏移或响应延迟，同时也可用来限制 Token 消耗以降低模型费用。
 - （2）系统提示词：定义模型在当前会话中所扮演的角色或语境，用于引导对话处于特定风格或领域。例如可以设置为“你是一个心理咨询师”或“你是编程人员”。
 - （3）Temperature：控制生成文本的随机程度。数值越高，生成内容越具创意和不确定性；数值越低，内容则更确定、稳重，适合需要准确回答的场景。
 - （4）Top P：与 temperature 联合使用，可调节生成内容的多样性与可控性。
 - （5）Top K：用于控制从知识库中检索出的文档数量。Top K 越大，检索内容越丰富，但也可能引入噪声；Top K 越小，相关性更高，但可能遗漏有用信息。同时 Top K 也可以控制 Token 消耗。
 - （6）历史消息记录数：控制历史消息作为上下文输入的数量。适当缩减历史记录有助于提升响应效率，并控制 Token 消耗；提高历史消息记录数则有助于上下文完整。

2.3.4 知识库模块的功能需求

如图 2.3 所示，知识库模块是对话系统中关键的数据支撑模块，主要负责用户用于构建和管理其私有知识内容。该模块由“知识库管理”和“文件管理”两大子模块构成。

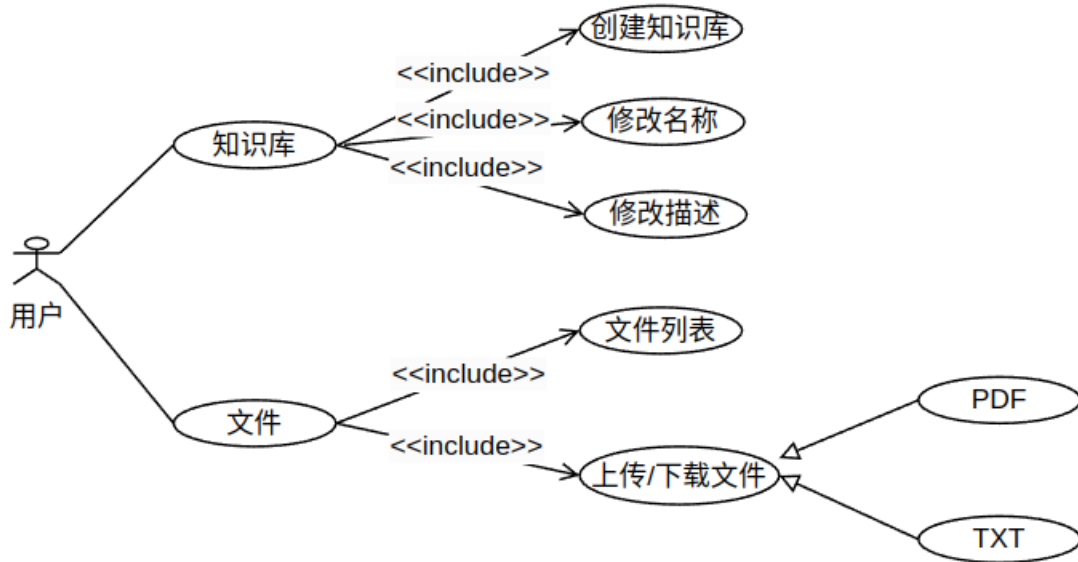


图 2.3 知识库模块用例图

1. 知识库管理

(1) 创建知识库：用户可以新建多个知识库，用于管理不同主题或项目的知识内容。

(2) 修改名称：支持用户对已创建的知识库进行重命名，便于后续识别与管理。

(3) 修改描述：用户可以为每个知识库添加或编辑描述信息，用于标明用途、内容来源或其他说明。

2. 文件管理

(1) 文件列表：展示知识库中所有已上传的文件。

(2) 上传/下载文件：用户可将本地文件上传至指定知识库中，并可指定文本分块大小和重叠大小，系统将进行文本分片与向量化处理，用于后续的 RAG 检索。上传后，支持用户从知识库中下载原始上传的文件，便于备份与复用。

(3) 上传文件类型：上传文件类型支持 TXT 和 PDF，对于 PDF 文件，系统将自动进行 OCR 转换成文本文件。

2.3.5 模型配置模块的功能需求

如图 2.4 所示，模型配置模块是系统管理员用于管理大模型服务接入信息的重要功能组件，主要包括模型的新增、修改、激活与关闭等操作，确保系统能够灵活接入不同的大模型服务。

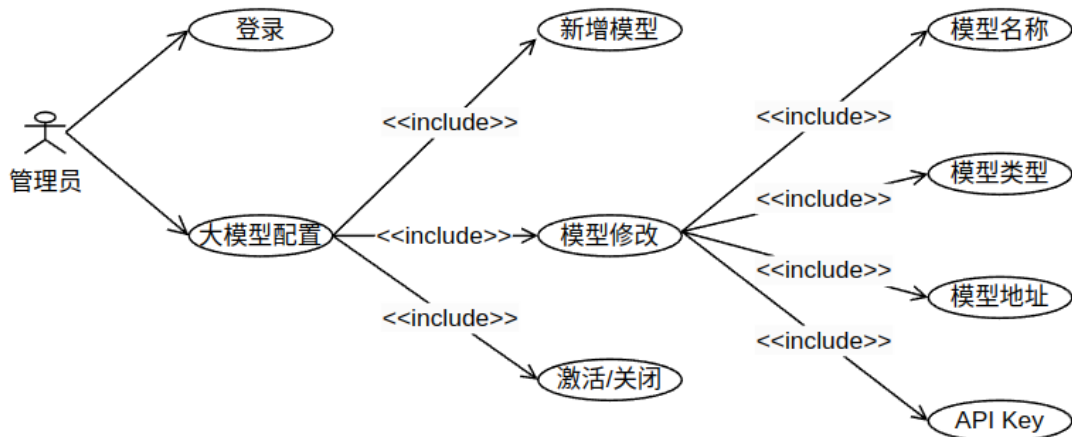


图 2.4 模型配置模块用例图

1. 登录功能：管理员通过输入管理账户和密码进入模型配置页面。
2. 新增模型：管理员可通过新增模型功能，向系统中添加一个新的大模型服务接口。在新增过程中，需要填写模型名称，模型类型（文本生成或图文理解），模型地址（可选）和 API Key。
3. 模型修改：当已接入的模型配置发生变更时，如 API 地址或访问密钥更换，管理员可通过该功能对现有模型信息进行编辑。
4. 模型激活/关闭：为保证服务稳定运行，管理员可以手动控制某一模型的“激活”或“关闭”状态，处于激活状态的模型将被系统用于实际对话服务，关闭状态下的模型不会被调用，可用于停用调试或临时下线。

2.4 系统非功能性需求

在需求分析中，虽然功能性分析很重要，但是像可用性、可扩展性、系统性能以及可维护性等非功能性分析也同等重要。

可用性主要体现在系统能够保持稳定运行，系统界面简洁，视觉层次清晰，交互流畅度能够得到保证^[12]。

可扩展性主要指的是跟随系统之后的业务需要，系统应支持二次开发^[13]。系统架构设计和代码组织结构应注重高内聚低耦合的特性，以保证未来在二次开发时系统架构不会有太大的改动。例如在本系统中，作为一个基于大模型的智能对话系统，当前主要集成了国内主流厂商的大模型 API 接口（如豆包、Moonshot、DeepSeek 和通义），但随着国内外大模型市场的不断发展，未来可能需要接入不同厂商的模型服务，甚至支持私有化部署的本地模型。为了实现这一目标，系统在设计时对大模型调用部分进行了接口抽象与模块封装，通过统一的模型适配层来实现模型调用逻辑的解耦。

系统性能主要指的是系统响应时间、系统支持的并发量等^[14]。在智能对话系统中，响应时间是性能评估的核心指标之一，直接影响用户交互的流畅度与使用体验。用户输入问题后通常希望能够在尽可能短的时间内得到准确、完整的回答。因此，系统应尽可能降低输入到输出之间的延迟时间，确保响应过程快速、连续、不卡顿。

可维护性主要在与系统的运维难度。在系统运行过程中可生成相应的日志记录，方便后期随时查询。开发时对代码进行注释，针对系统故障或者用户操作不当造成的问题，可以短时间内发现并修复 bug，有利于后期系统维护。

2.5 本章小结

本章从系统环境、技术选型、功能需求和非功能性需求四个方面进行了全面分析。系统环境从开发环境、测试环境和部署环境展开；技术选型从语言和框架、数据存储和大模型三个方向展开；功能上围绕用户登录、对话管理、知识库构建与模型配置展开，为系统设计与开发提供了明确依据。

第三章 智能对话系统的概要设计

3.1 系统的架构设计

智能对话系统采用 B/S 架构^[15]进行设计，使用 Nginx 来实现反向代理。反向代理是指代理服务器接收客户端的请求，将请求转发给真实服务器后获取返回结果，再将返回结果反馈给客户端的过程^[16]。此外，还会使用 OSS 对象存储进行文件和图片存储，使用 VikingDB 向量数据库构建文档索引，使用多个大模型提供 API 服务。最终，该系统的服务器部署如图 3.1 所示。

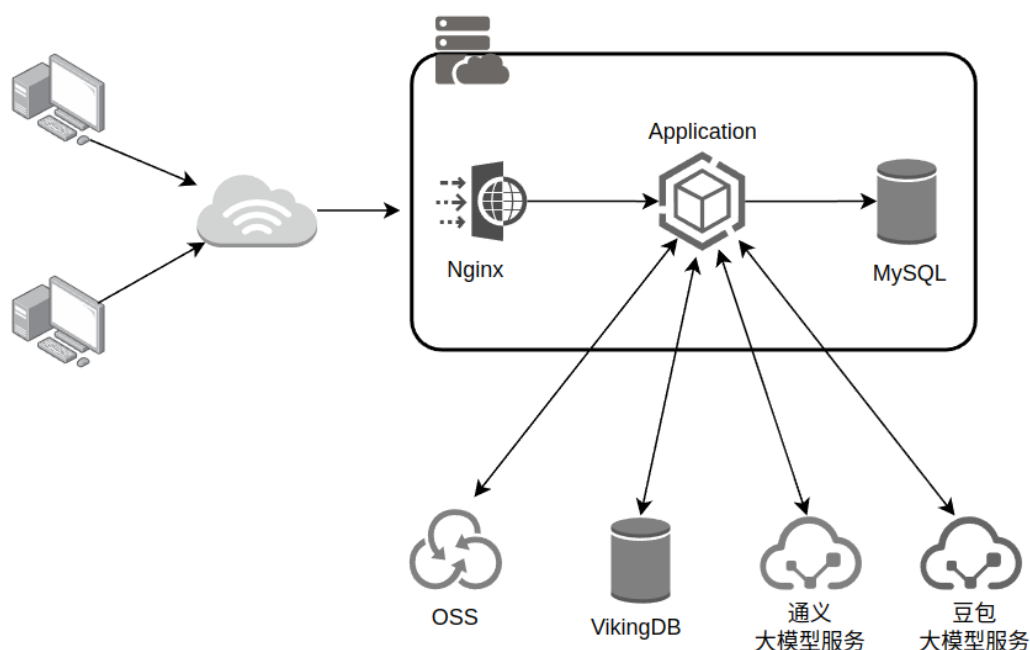


图 3.1 部署架构

整个对话系统采用前后端分离的技术架构，自上而下主要分为五层，最上层为视图层、其次为控制层、然后是业务层、再然后是仓储层、最后一层是数据层^[17]，整体的技术架构如图 3.2 所示。

视图层主要使用了 Vue、HTML、CSS 等相关技术，其中 Vue 主要是处理页面相关的逻辑，HTML 主要用于定义页面中相关的元素，CSS 主要是为页面添加相应的样式。

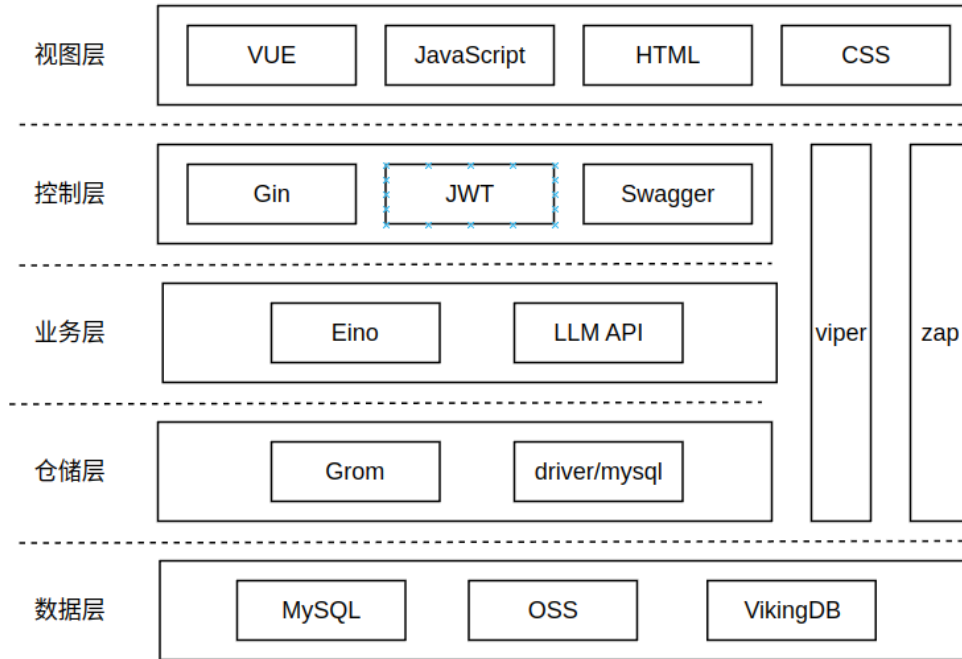


图 3.2 技术架构

控制层主要使用 Swagger 生成接口文档；使用 Gin 接收与解析 HTTP 请求，进行参数校验并路由到相应的业务处理模块；使用 JWT 实现用户身份认证与接口访问控制。

业务层主要负责业务逻辑的处理，使用 Eino 框架构建对话、建立嵌入索引和检索增强等核心业务流程，简化了大模型接入与会话管理。

仓储层封装了对数据库的访问逻辑，使用 Gorm 框架简化了 SQL 操作，通过模型映射方式实现数据库表的增删改查。所有与业务相关的数据均通过此层进行访问，包括用户信息、知识库、文件、会话消息等。

数据层主要使用了关系型数据库 MySQL 对系统的数据进行存储，使用 OSS 对象存储对文件、图像进行存储，使用 VikingDB 对文档建立索引和检索。

此外，系统中还引入了以下中间件组件以提升系统能力：使用 Viper 作为配置中心，支持系统配置的集中管理与热加载；使用 Zap 高性能日志库用于系统运行过程中的关键日志记录与异常排查。

3.2 系统的模块设计

经过前面需求分析和架构设计之后，智能对话系统主要分为四个模块，分别

是登录模块、会话模块、知识库模块以及模型配置模块。系统的总体模块功能图如图 3.3 所示。

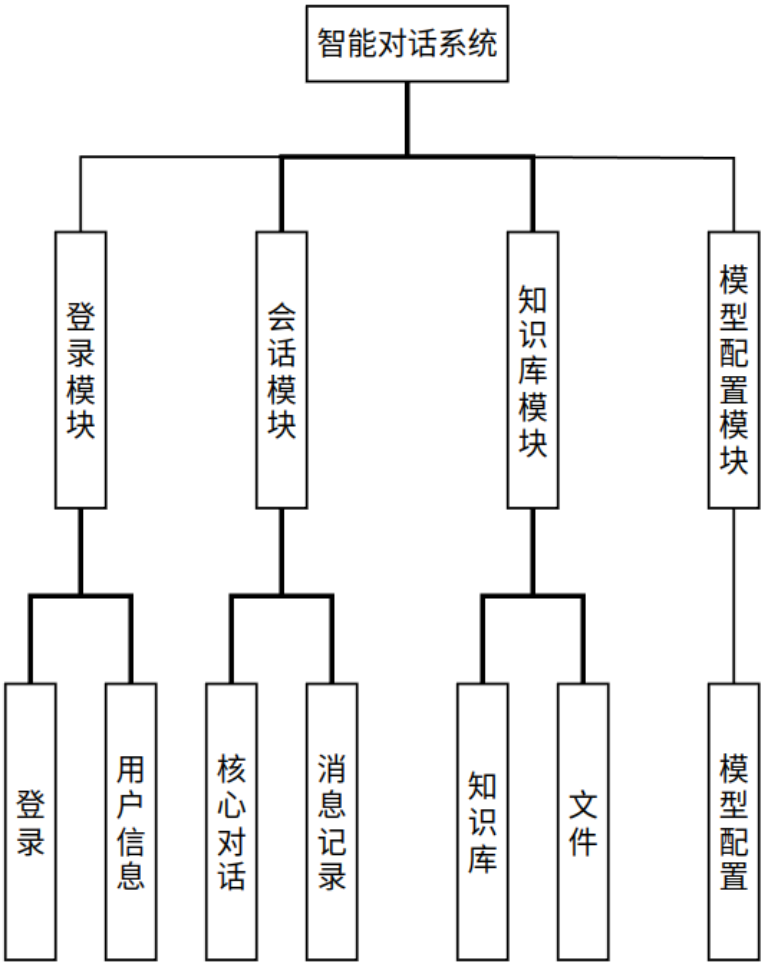


图 3.3 模块层次图

登录模块主要涉及登录和用户信息两个子模块，登录包括 OAuth2.0 三方授权登录和 JWT 鉴权和状态保持；用户信息主要是用户信息的管理。会话模块包括核心对话和消息记录两个子模块，核心对话涉及会话管理，会话动态配置和会话过程与大模型的交互。知识库模块分成知识库管理和文件管理两个子模块。模型配置模块主要是模型配置。

3.3 数据库设计

3.3.1 数据库 ER 图

根据上一章的功能性需求分析，对话系统中主要涉及五个实体，分别是用户、

知识库、会话、消息和模型。它们之间的关系分为三种：一对一、一对多和多对一[18]。

数据库 ER 如图 3.4 所示。

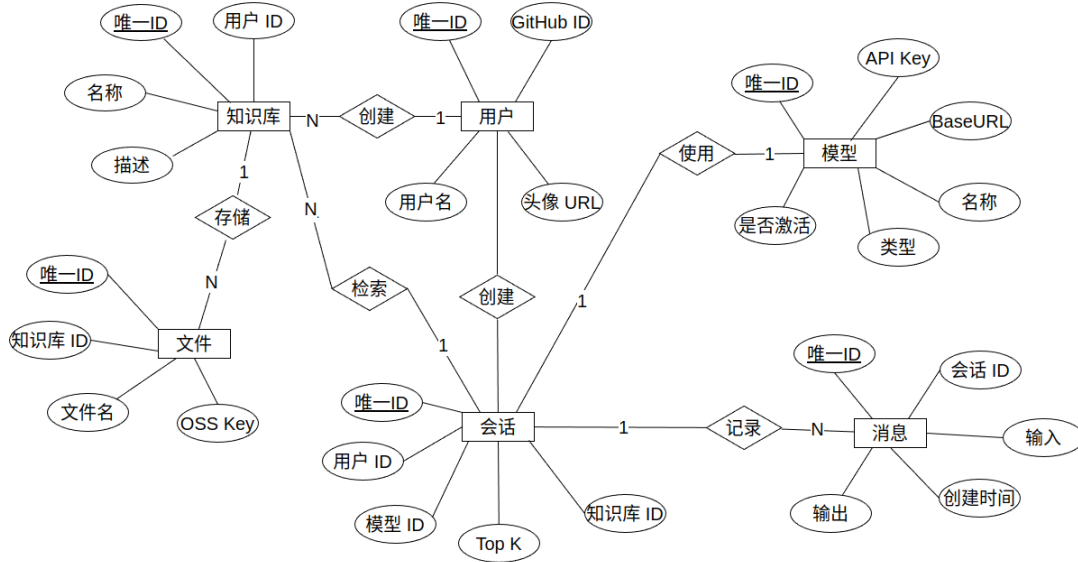


图 3.4 数据库 ER 图

3.3.2 数据库表设计

(1)如表 3.1 所示,用户信息表主要存储用户相关信息,包括用户 ID、Github 三方 ID、户名和头像链接,其中,用户 ID 是主键。

表 3.1 用户信息表

字段名	字段描述	数据类型	长度	是否主键
user_id	用户唯一 ID	Bigint	20	Y
external_id	Github 三方 ID	Bigint	20	N
name	用户名	Varchar	255	N
avatar_url	头像链接	Varchar	255	N

(2)如表 3.2 所示,会话表存储会话元信息和配置信息,主要包括会话 ID、用户 ID、使用模型 ID、会话名称等字段,会话动态配置主要包括历史消息记录数和知识库 ID 列表等字段。其中,会话 ID 是主键。

表 3.2 会话表

字段名	字段描述	数据类型	长度	是否主键
conversation_id	会话唯一 ID	Bigint	20	Y
title	会话标题	Varchar	255	N
user_id	用户唯一 ID	Bigint	20	N
model_id	模型 ID	Tinyint	8	N
	指定一个或多个			
knowledge_base_ids	知识库，格式： "123-124"	Varchar	255	N
history_count	使用历史消息作 为上下文的条数	Tinyint	8	N
...	...	...	...	...

(3) 如表 3.3 所示，消息记录表主要存储会话的所有历史消息记录，主要包括消息 ID、对话 ID、用户消息内容、回答消息内容和创建时间。其中，消息 ID 是主键。

表 3.3 消息记录表

字段名	字段描述	数据类型	长度	是否主键
message_id	消息唯一 ID	Bigint	20	Y
conversation_id	对话唯一 ID	Bigint	20	N
input	用户消息内容	TEXT	65,535	N
output	回答消息内容	TEXT	65,535	N
created_at	创建时间	TIMESTAMP	4	N
...	...	...	...	...

(4) 如表 3.4 所示，知识库表主要存储知识库元信息，主要包括知识库 ID、知识库名称、知识库描述和用户 ID 字段。其中，知识库 ID 是主键。

表 3.4 知识库表

字段名	字段描述	数据类型	长度	是否主键
knowledge_base_id	知识库唯一 ID	Bigint	20	Y
name	知识库名称	Varchar	255	N
description	知识库描述	Varchar	1024	N
user_id	用户唯一 ID	Bigint	20	N

(5) 如表 3.5 所示，模型表主要存储模型的元信息，主要包括模型 ID、模型名称、API KEY、URL、状态和模型类型。其中，模型 ID 是主键。

表 3.5 模型表

字段名	字段描述	数据类型	长度	是否主键
model_id	模型唯一 ID	Tinyint	8	Y
name	模型名称	Varchar	255	N
api_key	鉴权	Varchar	255	N
base_url	调用模型 URL	Varchar	1024	N
status	状态（0 未激活，1 激活）	Tinyint	8	N
model_type	模型类型	Tinyint	8	N

3.4 本章小结

本章主要介绍了智能对话系统的整体概要设计，包括系统的架构设计、模块划分以及数据库设计。在架构上，系统采用前后端分离的 B/S 架构，集成多种中间件与大模型 API 提供智能对话服务；在功能上，将系统划分为登录、会话、知识库和模型配置四大模块；在最后，采用 ER 图展示了数据库表的设计。为后续系统详细设计与实现奠定了基础。

第四章 智能对话系统的详细设计

4.1 登录模块

4.1.1 OAuth2.0 登录

登录模块主要涉及用户登录和用户信息获取。用户登录采用 OAuth2.0 协议进行授权认证。OAuth2.0 是目前主流的开放授权标准，广泛应用于第三方登录场景，如 GitHub、微信等。系统使用 GitHub 的 OAuth 应用作为认证提供方，完整的登录时序图如图 4.1 GitHub OAuth2.0 登录流程所示。

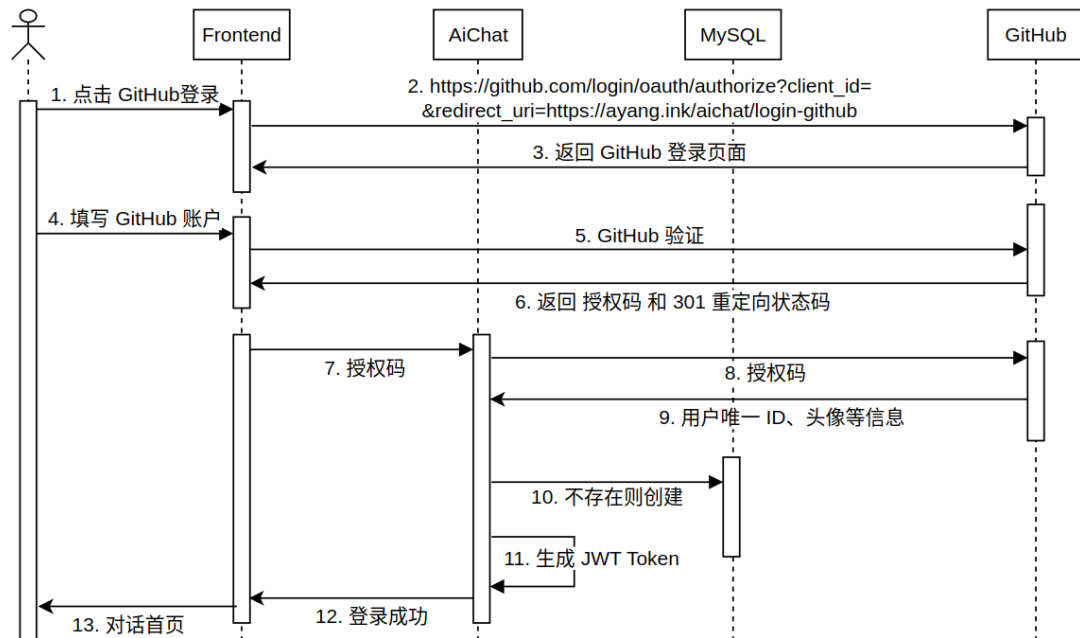


图 4.1 GitHub OAuth2.0 登录流程

用户点击 GitHub 登录将跳转至 GitHub 授权页面，输入正确的账户密码后 GitHub 返回授权码，并重定向回智能对话系统，后端使用该授权码请求 GitHub 获取用户唯一 ID 和头像等信息，根据 GitHub 用户 ID 查找数据库，如果不存在则表示为系统新用户，会创建记录保存用户信息。最后，根据用户信息生成 JWT token，并返回给前端，完成登录。

4.1.2 JWT 登录验证和状态保持

为实现后续的用户会话管理和接口访问控制,引入 JWT 作为身份凭证。JWT 是一种轻量级、无状态的身份验证机制,由三部分组成: Header、Payload 和 Signature。系统在完成 OAuth2.0 登录流程后,后端会生成一个包含用户身份信息的 JWT 并签名,该 Token 将通过 HTTP 响应返回给前端。前端将该 Token 缓存至 LocalStorage,后续每次发起请求时都在请求头中携带此 Token。

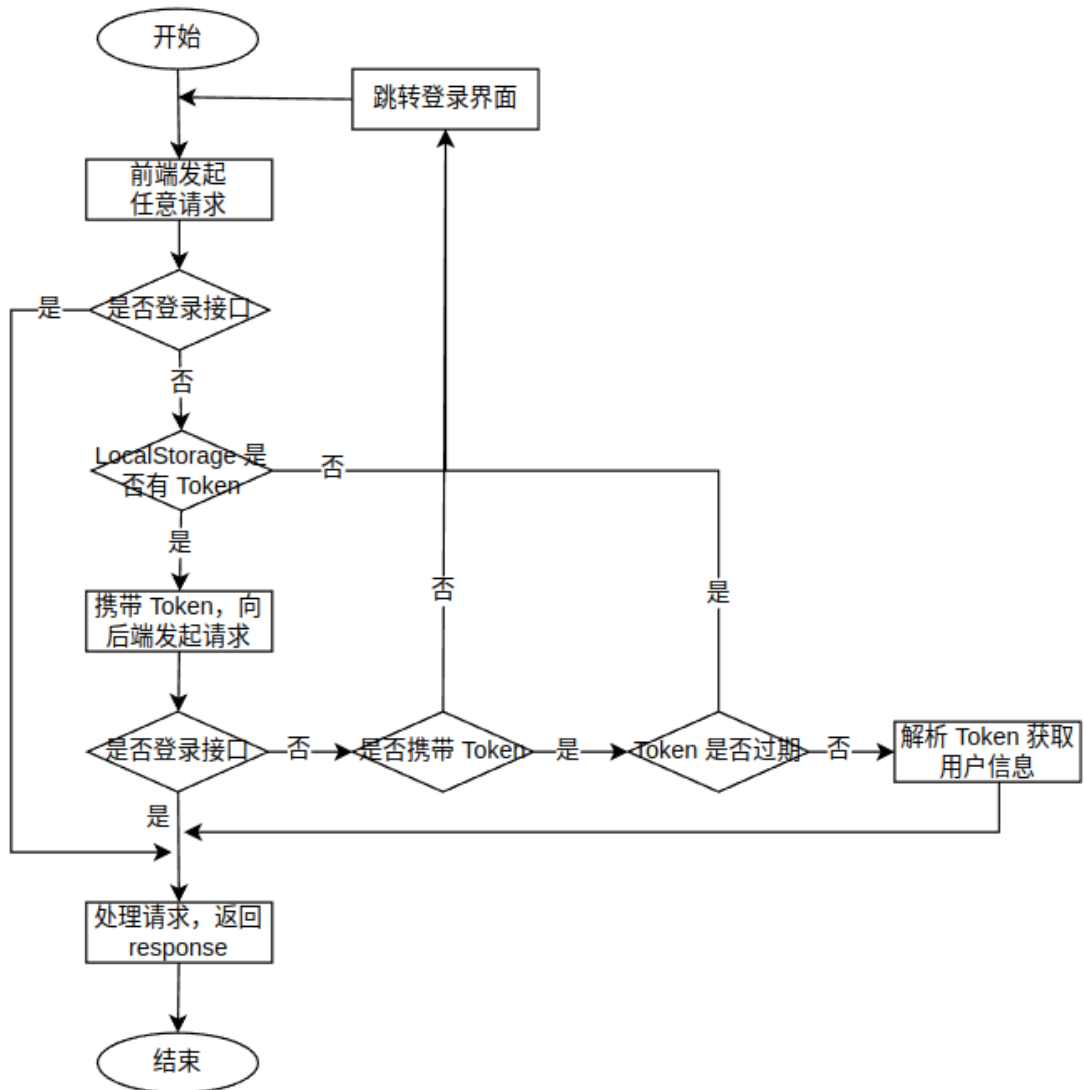


图 4.2 JWT 登录验证流程

如图 4.2 JWT 登录验证流程前端请求任意接口, 经过前端拦截器和后端中间件解析和校验 JWT 的完整流程, 即可实现用户身份验证和自动登录即登录状态保持两项功能。前端发起任一请求前, 会判断是否是登录接口, 如果是登录接

口，会判断 `LocalStorage` 是否缓存有 `token`，没有则跳转到登录页面，有则携带 `token` 向后端发起请求。后端拦截器同样会先判断是否登录接口，如果是非登录接口，会进行登录验证流程。先判断是否携带 `token`，没有则返回错误码，前端识别错误码后跳转到登录页面。再判断 `token` 是否过期，过期同样返回错误码。最后解析并验证 `token` 的合法性，验证通过，则鉴权成功，进入处理请求的流程。

4.2 会话模块

4.2.1 对话流程图

图 4.3 展示了用户从进入系统开始，到最终获得对话结果的整个流程。整个对话流程可分为以下几个阶段：

1. 用户身份验证。首先判断用户是否已登录。若未登录，则跳转至登录页面完成身份认证；若已登录，则进入会话首页。
2. 会话配置修改。在会话页面中，用户可选择是否修改会话配置，如选择会话类型、指定模型、温度值、是否启用历史消息、启用历史消息数量、是否开启 RAG 等配置。用户完成设置后，点击保存配置将变更保存到数据库中，下次会话立刻生效。
3. 用户输入对话内容。
4. 系统会进一步判断输入是否包含图像信息，包含则上传图像到 OSS 并获取访问图像的 URL。接下来会判断是否启用 RAG 功能，若启用，则将根据用户输入，结合当前绑定的知识库，对其进行语义检索，从中提取 Top K 篇相关文档，用于后续生成回答；然后，系统会判断是否启用历史消息上下文功能，若启用，将从数据库中提取当前用户指定数量的对话历史消息记录。
5. 综合用户输入、图像 URL、知识库文档、历史消息等信息，格式化构造 Prompt，并将其发送至 LLM API。
6. 接收到大模型的响应后，系统采用 SSE 方式将响应内容实时推送至前端，模拟打字式的对话体验。

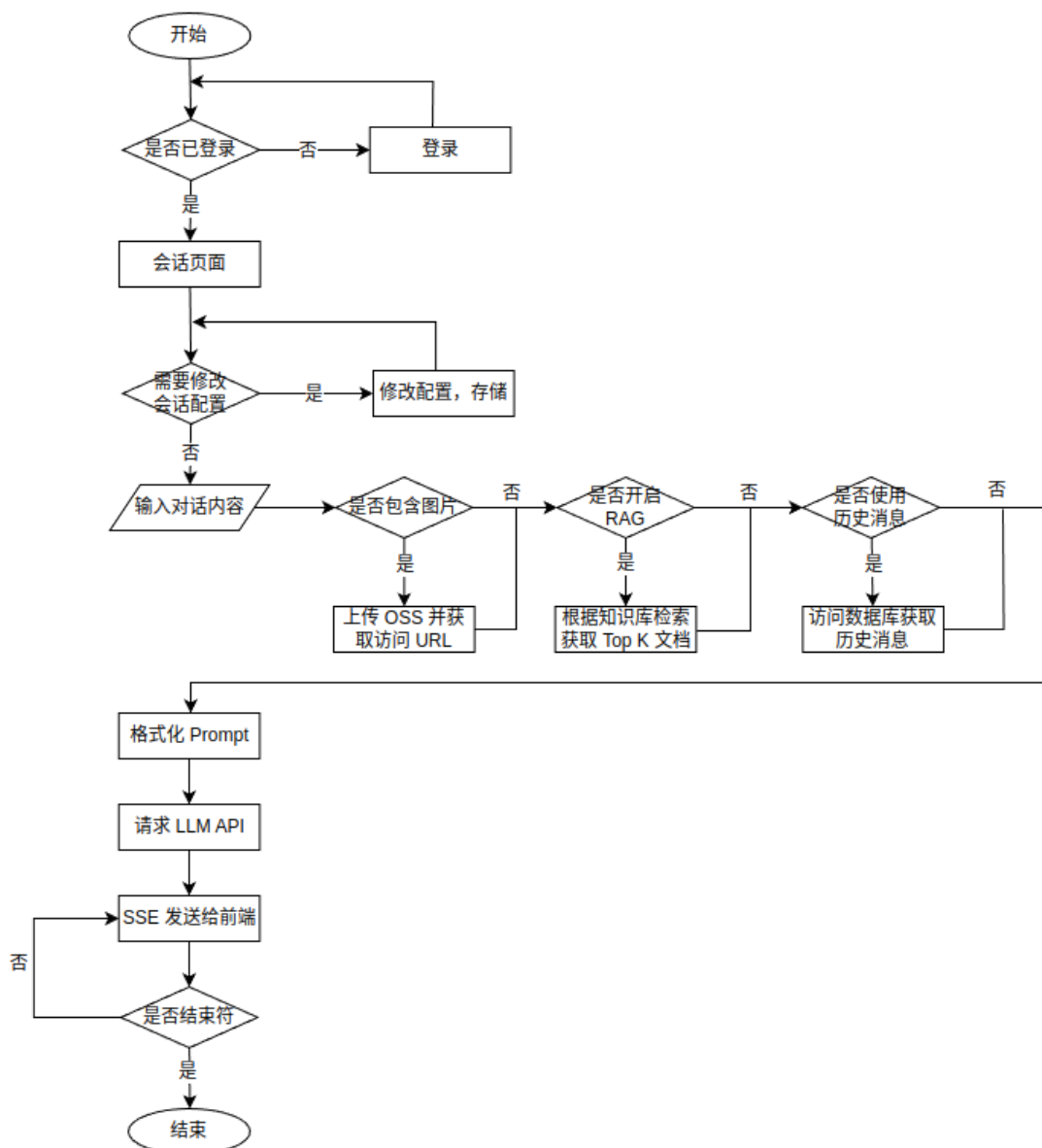


图 4.3 对话处理流程

如图 4.4 所示，为提升对话系统中与大模型交互的实时性体验，对话接口前后端通信采用了 SSE 作为流式响应的技术方案。传统的 HTTP 请求-响应模型为一次性返回所有响应数据，而大模型生成回答的过程通常是逐 Token 生成的，若使用普通请求方式，用户需等待整个响应结束才能看到结果，交互延迟显著。在智能对话系统中，大模型输出结果采用流式接口，后端通过 Gin 框架构建 SSE 响应流，前端基于 Vue3 接收并逐步渲染模型输出内容，提供高效、自然的用户交互体验。



图 4.4 对话流式转换逻辑

4.2.2 删除会话流程图

如图 4.5 所示，其展示的是删除会话的完整流程图。

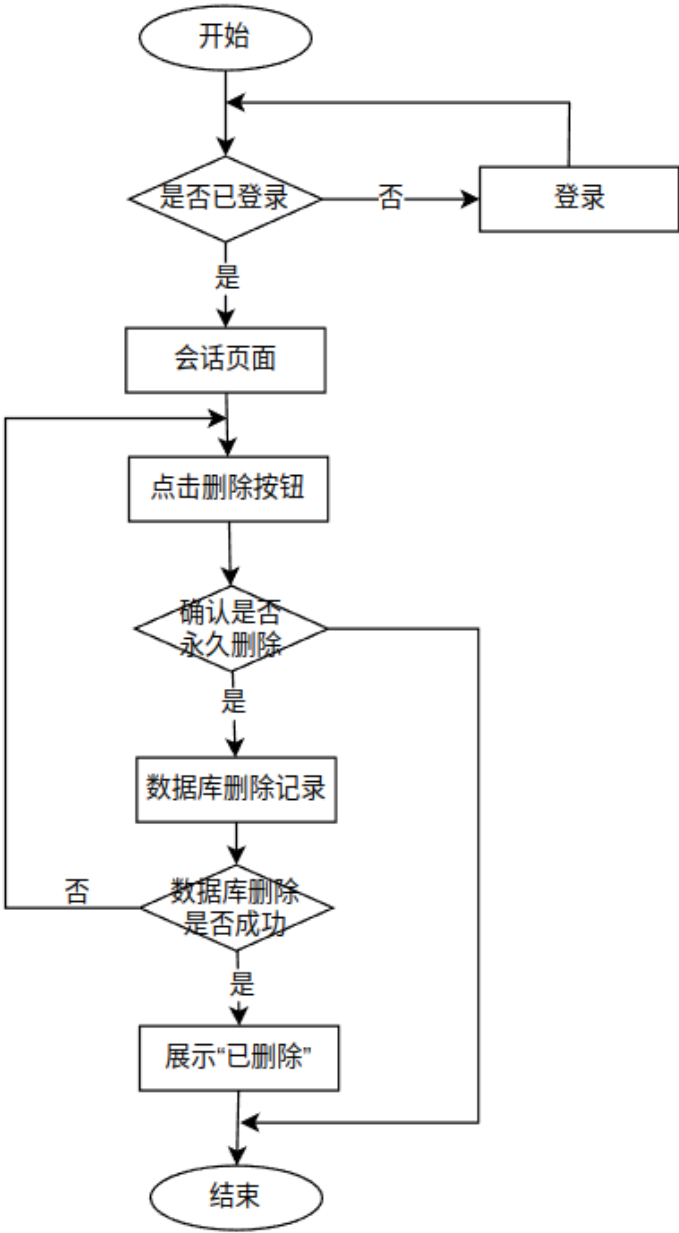


图 4.5 删除会话处理流程

首先判断用户是否已登录。若未登录，则跳转至登录页面完成身份认证；若

已登录，则进入会话首页。点击删除按钮，会弹出提醒框再次确认是否永久删除该会话，如果点击“是”则调用后端接口尝试删除对应的数据库记录，删除成功则页面提示“已删除”，失败则展示“删除失败，请重新删除”。

4.3 知识库模块

4.3.1 上传文件

如图 4.6 所示，其展示的是上传文件并且建立嵌入向量索引的时序图。

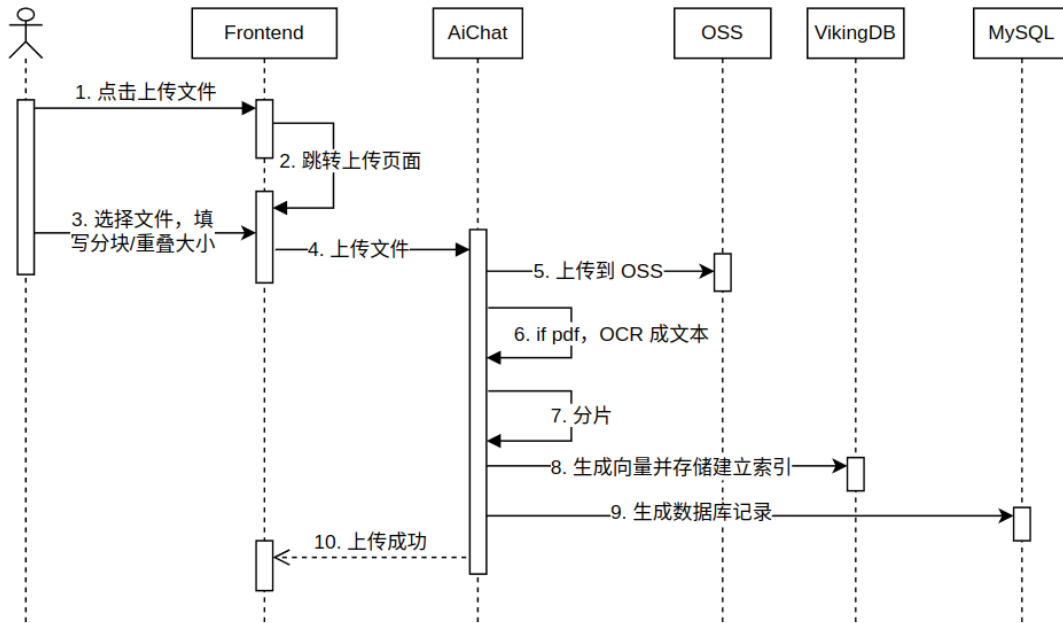


图 4.6 知识库上传文件处理流程

用户点击“上传文件”按钮，系统跳转至上传页面，选择本地文件，并填写文件的分块大小及重叠大小参数，点击“确认上传”按钮，前端将所选文件提交给后端服务。后端接收到上传请求后，将文件转发至对象存储服务 OSS 进行存储。若文件类型为 PDF，系统会调用 OCR 模块将其转化为可处理的文本内容。然后文本内容过根据指定分块大小和重叠大小被切分为多个片段，用于后续向量生成。系统调用向量化服务，将文本片段转换为向量，存储至 VikingDB，同时建立索引以支持后续检索。最后向 MySQL 数据库写入文件元信息与处理状态，便于用户查询和管理。当其中有任一操作失败时，都会返回给前端失败原因进行展示。所有处理环节执行完毕后，向前端返回“上传成功”响应，用户界面提示上传完成。

4.3.2 下载文件

如图 4.7 所示，其展示的是下载知识库文件的时序图。

用户打开知识库详情页，点击下载按钮。前端会发送第一个请求到后端，后端先根据文件唯一 ID 查找数据库，拿到该文件存储在 OSS 的 bucket key，再根据 bucket key 请求火山云 OSS 服务，生成该文件经过签名的下载 URL，过期时间设置为 30 分钟，然后将该 URL 返回给前端。前端根据 URL 向火山云 OSS 服务发起下载请求，下载完毕后浏览器会弹框展示文件，用户可以进行打开。

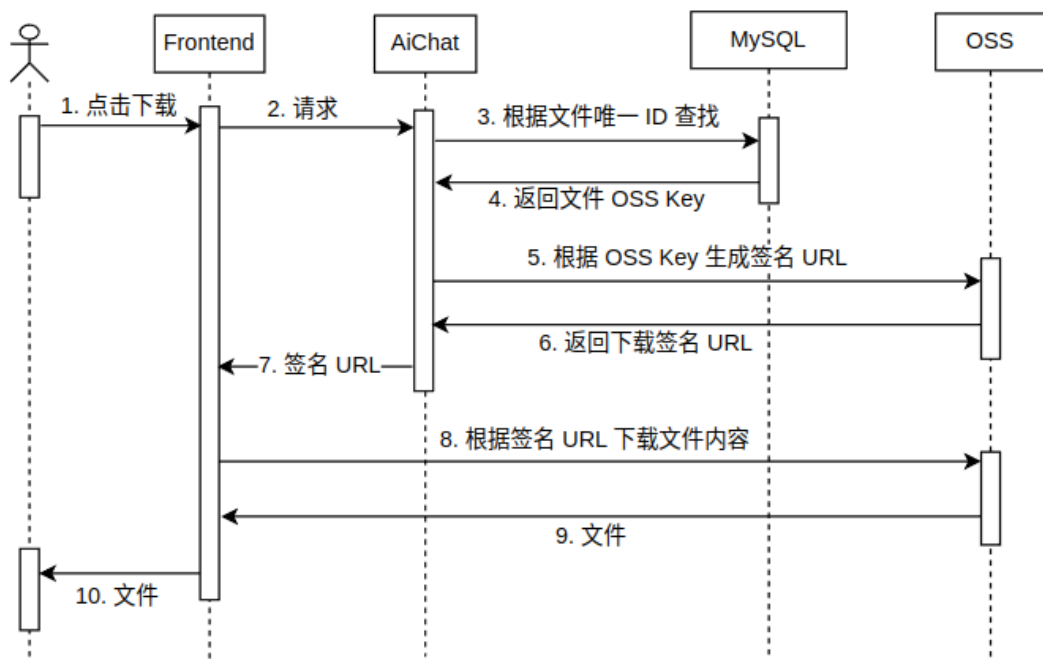


图 4.7 知识库下载文件流程

4.4 模型配置模块

如图 4.8 所示，其展示的是模型配置模块的流程图。

管理员输入管理员账号和密码，校验信息是否正确，如果错误给出提示返回失败，正确则进入首页。点击配置模型进入模型配置页面。可以进行新增模型、修改模型信息或激活/关闭模型等操作。点击“增加模型”会弹出模型编辑窗口，填写模型信息。新增模型在前端需要验证模型信息是否正确，必填字段是否完备等。点击提交触发检测，如果非法，会进行弹出提示并禁止提交。待再次修改后

可再次点击提交。新增或修改的记录会通过 form 表单的形式以 Post 请求的方式提交到后端，后端同样会进行模型信息合法性校验，通过后会进行数据库的插入或修改操作。

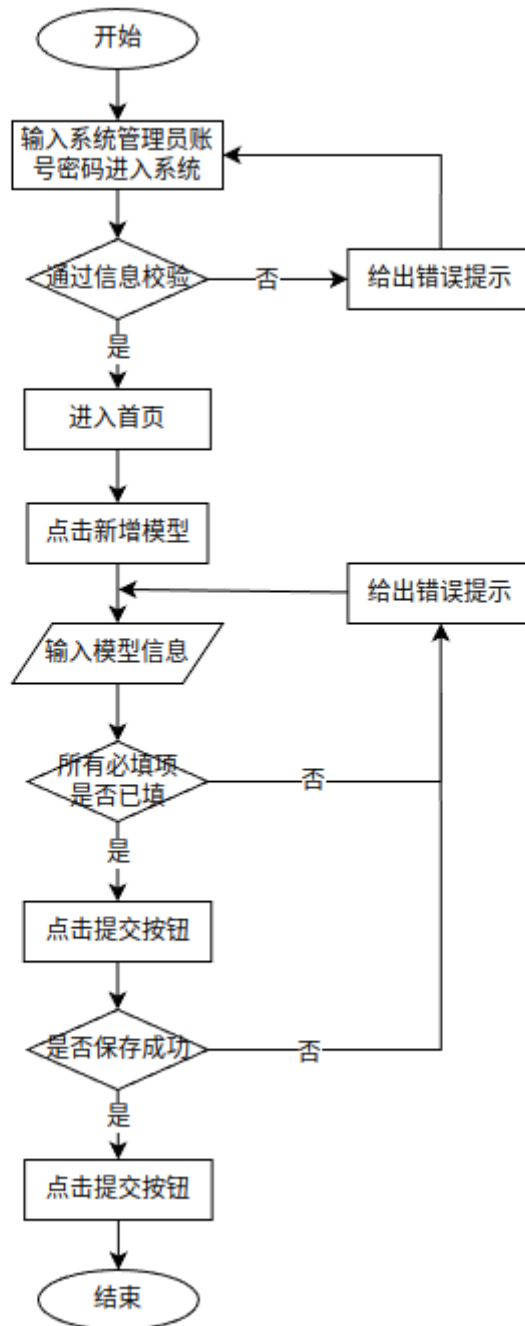


图 4.8 新增模型流程

4.5 本章小结

本章从系统的四个核心功能模块出发，分别对登录模块、会话模块、知识库

模块和模型配置模块进行了详细设计说明。重点阐述了各模块的业务流程、时序关系及关键技术实现,包括 OAuth 登录认证、JWT 身份验证、对话流程中 RAG 和 SSE 的应用、知识库的向量索引构建流程以及模型配置的操作逻辑。

第五章 智能对话系统的实现和测试

5.1 登录模块的实现

如图 5.1 所示，点击左图 GitHub 标识可以跳转到 GitHub 三方登录页面进行登录；管理员可以输入账号密码后点击“登录”按钮直接进行登录。

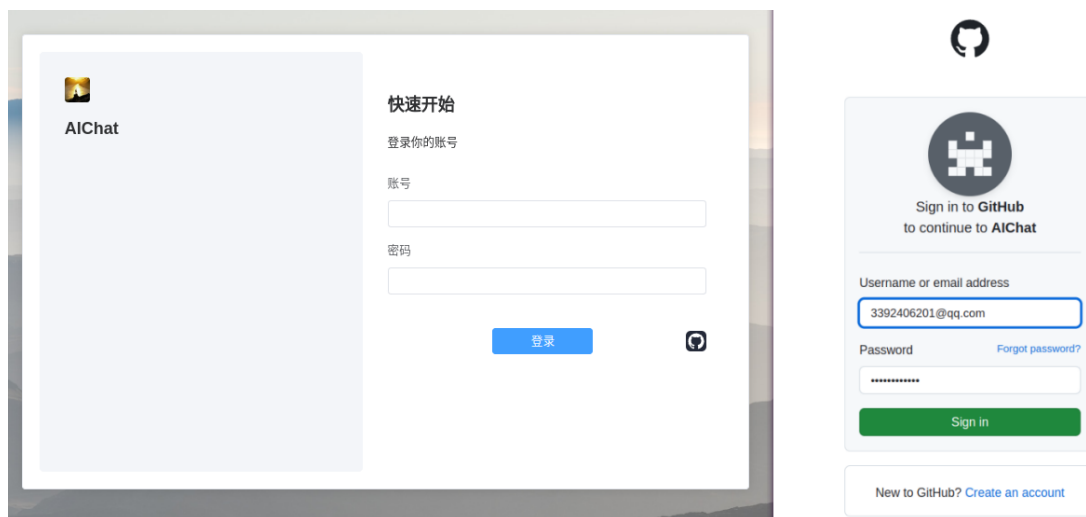


图 5.1 登录页面

图 5.2 为登录后进入核心会话页面，点击“退出登录”按钮即可退出登录状态。

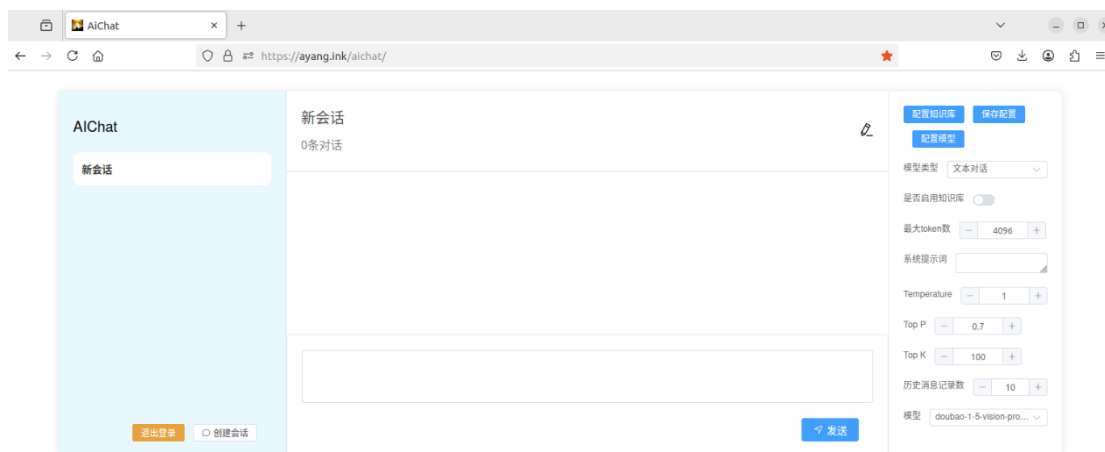


图 5.2 会话首页

5.2 对话模块的实现

5.2.1 会话操作

如图 5.3 所示，点击“创建会话”可以创建一个空会话。



图 5.3 会话操作

新建的空回话如图 5.4 所示，点击会话右上角，可以修改会话名称。

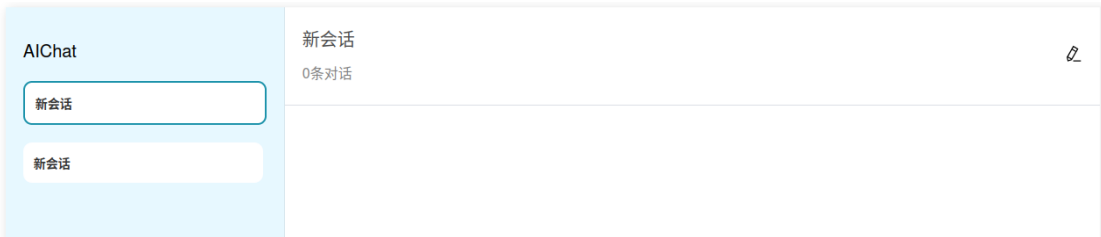


图 5.4 会话列表

如图 5.5 所示，修改会话名称成功会显示“修改成功”提示。

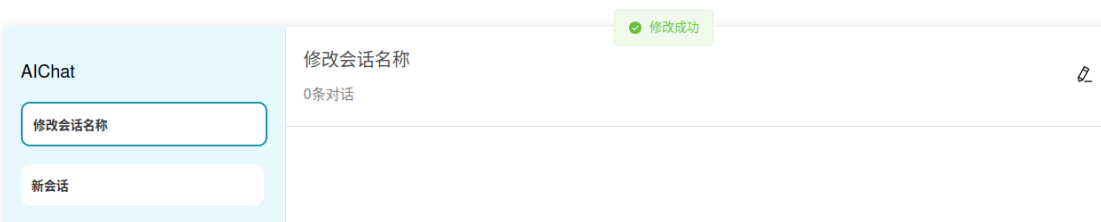


图 5.5 修改会话

如图 5.6 所示，鼠标滑动到左侧会话列表栏，会出现“×”符号，点击“×”可以删除会话。点击后会弹出确认框再次确认是否永久删除，点击“YES”按钮会永久删除会话，删除成功同样会显示“删除成功”提示。

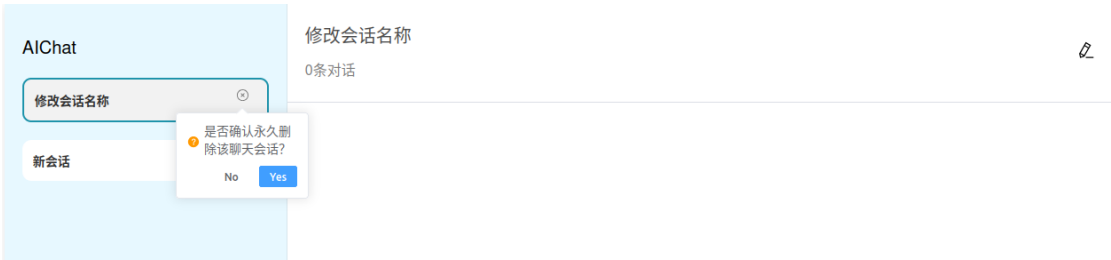


图 5.6 删除会话

5.2.2 修改会话配置

如图 5.7 展示了修改会话配置的页面。在配置页面上编辑，点击“模型类型”即可出现下拉选项，可以选择指定的模型；开启知识库后也可下拉出现所有知识库选项，可选择一个或多个知识库；系统提示词框可以输入文本 `prmopt`，最大 Token 数等可以通过点击“+”“-”或直接编辑输入。编辑完成后，点击“保存配置”按钮进行保存。

图 5.7 会话配置页面

如图 5.8 所示，修改模型类型为“图片理解”等配置后，点击“保存配置”，修改成功会展示“修改成功”提醒，同时“发送”按钮左侧出现“+”按钮可以点击上传图片。

5.2.3 对话功能

如图 5.9 所示，展示了核心对话页面，修改模型类型为“图片理解”，添加图

片，输入框输入问题。点击“发送”。

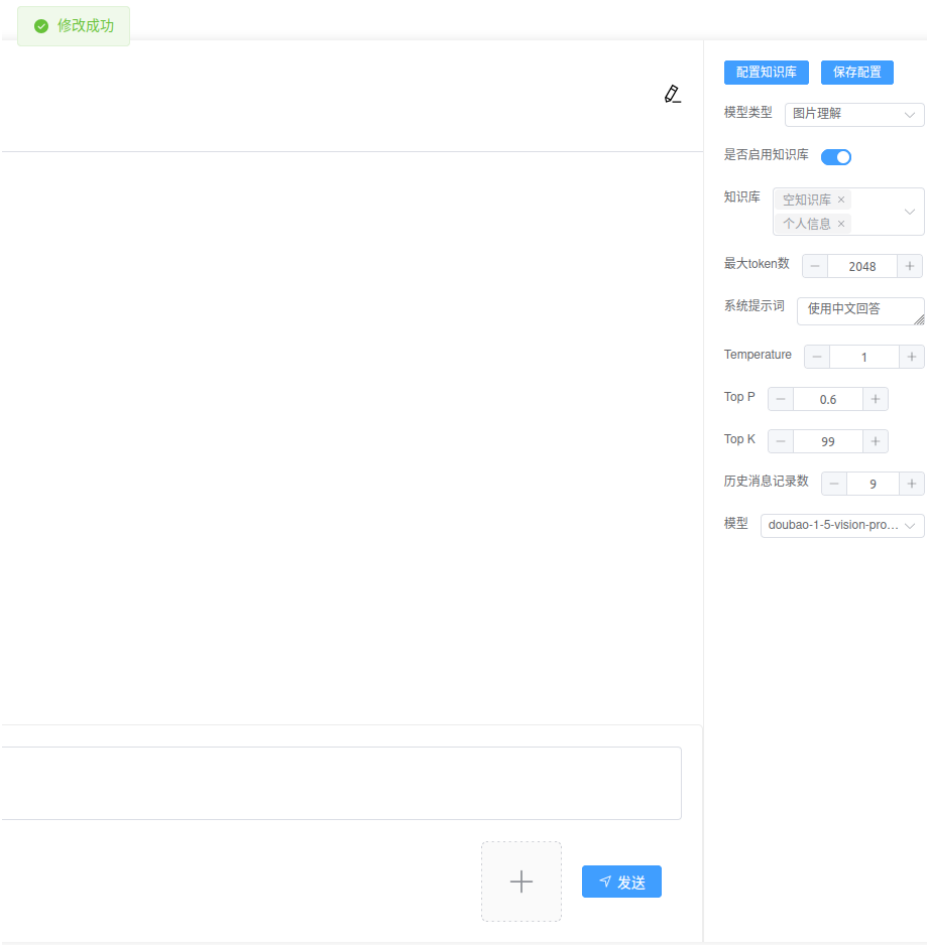


图 5.8 会话配置修改

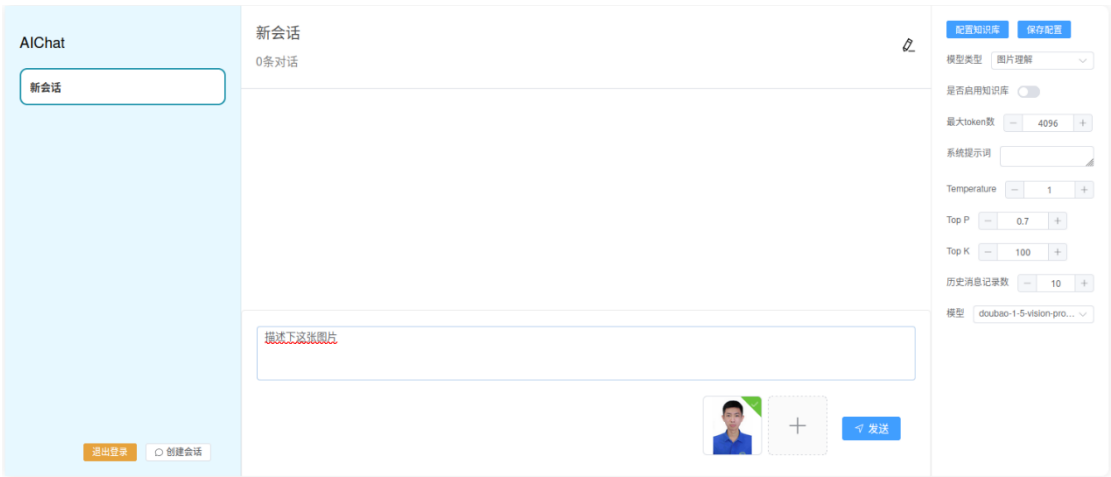


图 5.9 图片理解

如图 5.10 所示，展示了系统回答后的页面，会话框会显示该会话的所有历史记录。



图 5.10 图片理解回答

如图 5.11 所示，启用知识库 RAG 功能，知识库选择“个人信息”知识库，知识库含有个人简历的 pdf 文件。输入框输入“简述黄杨这个人”，根据个人简历回答问题。

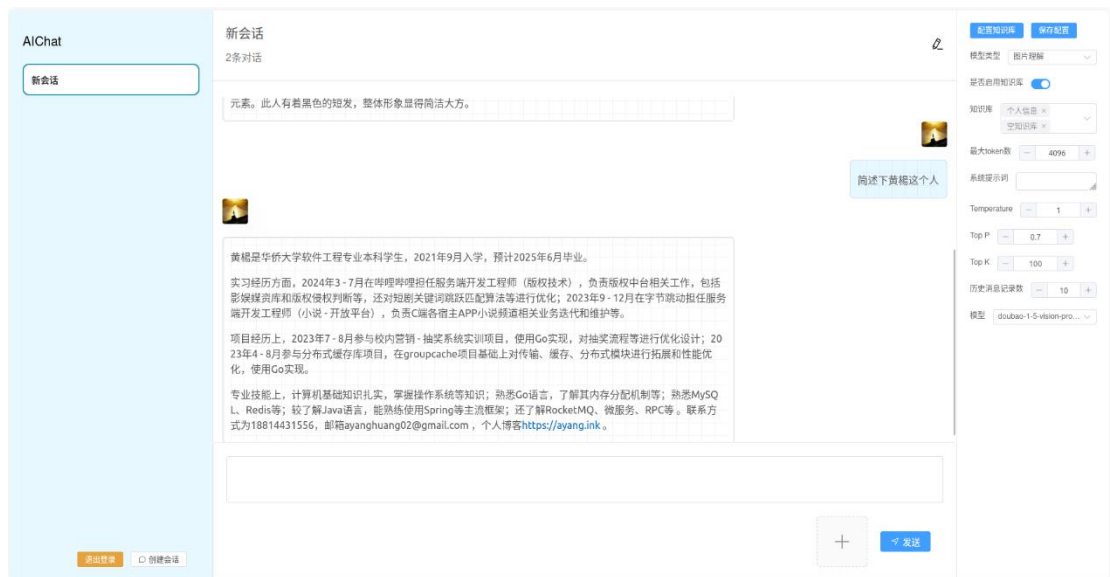


图 5.11 RAG 问答

5.3 知识库模块的实现

如图 5.12 所示，点击“创建知识库”，填写“名称”和“描述”，点击确认。

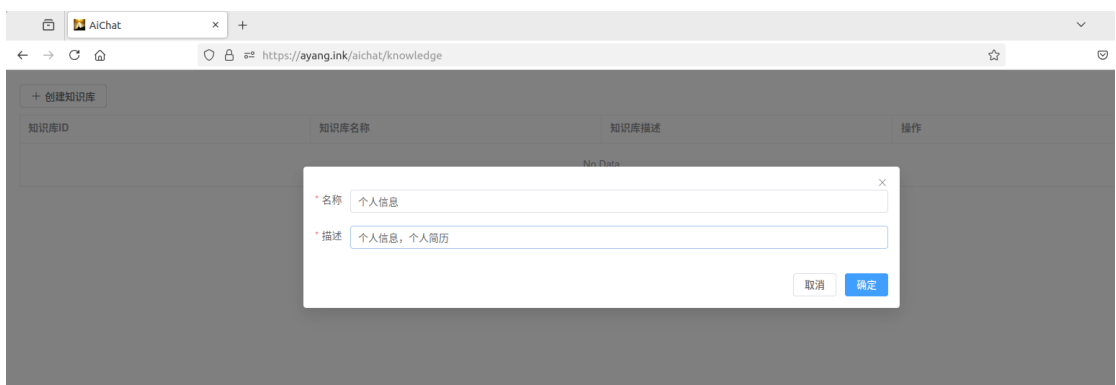


图 5.12 新建知识库

如图 5.13 所示，创建知识库成功会提示“创建成功”，知识库列表会展示新增的知识库。



图 5.13 新建知识库成功

如图 5.14 所示，点击知识库右侧上传文件的按钮，点击“上传文件”可打开电脑本地文件夹，选择文件。填写“分块大小”和“重叠大小”，点击确定。

如图 5.15 所示，上传成功后，点击知识库右侧编辑页面。可以展示文件列表，可以选择下载文件和删除文件。



图 5.14 上传文件

5.4 模型配置模块的实现

如图 5.16 所示，展示的模式配置页面，会列举系统中所有模型和其详细信

息。

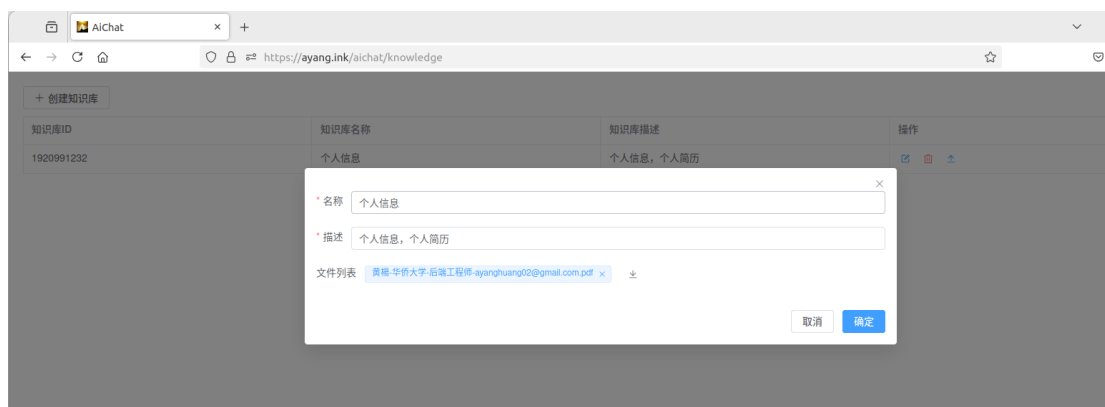


图 5.15 知识库编辑页面

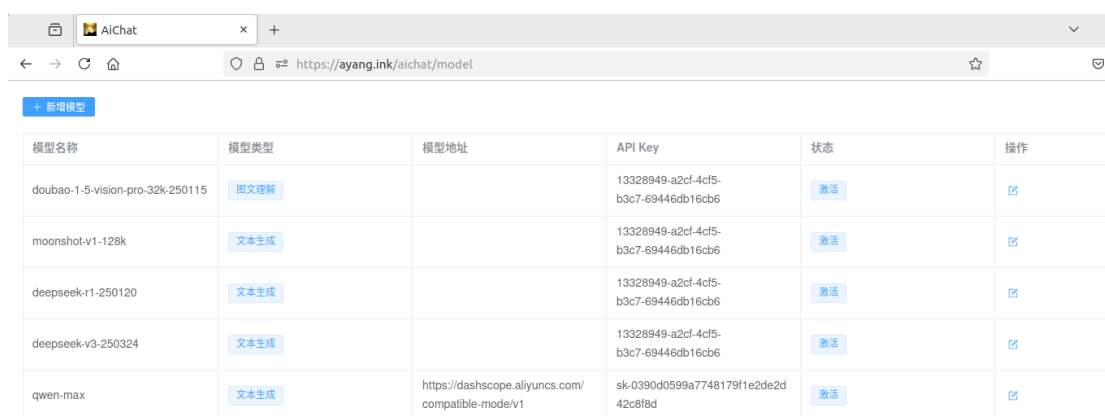


图 5.16 模型列表页面

如图 5.17 所示，点击“新增模型”按钮，弹出模型信息填写窗口，模型类型和状态可下拉选择，点击“提交”则发送创建请求。

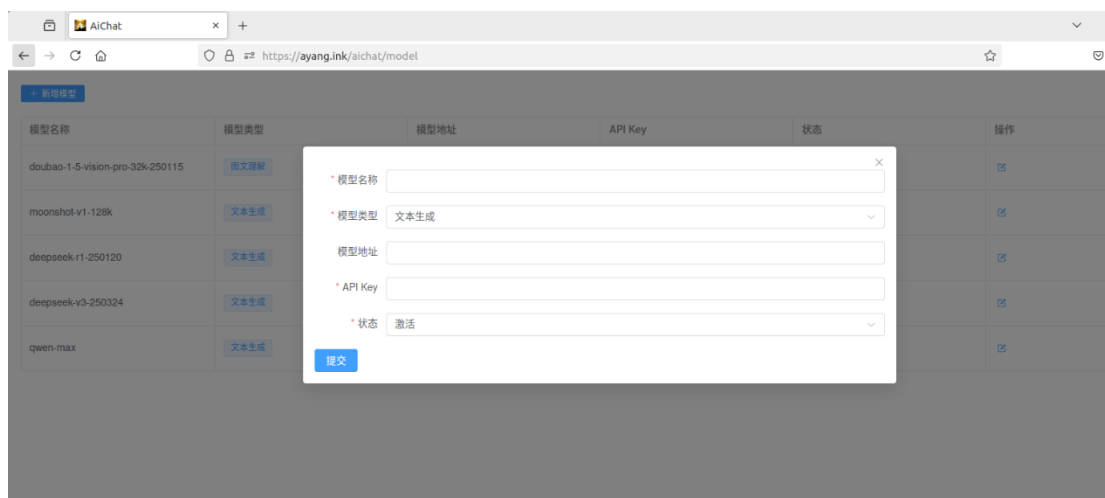


图 5.17 新增模型页面

如图 5.18 所示, 点击模型列表右侧编辑按钮, 可弹出模型信息编辑窗口, 点

击“状态”，会出现下拉选项，选择“未激活”，点击“提交”。

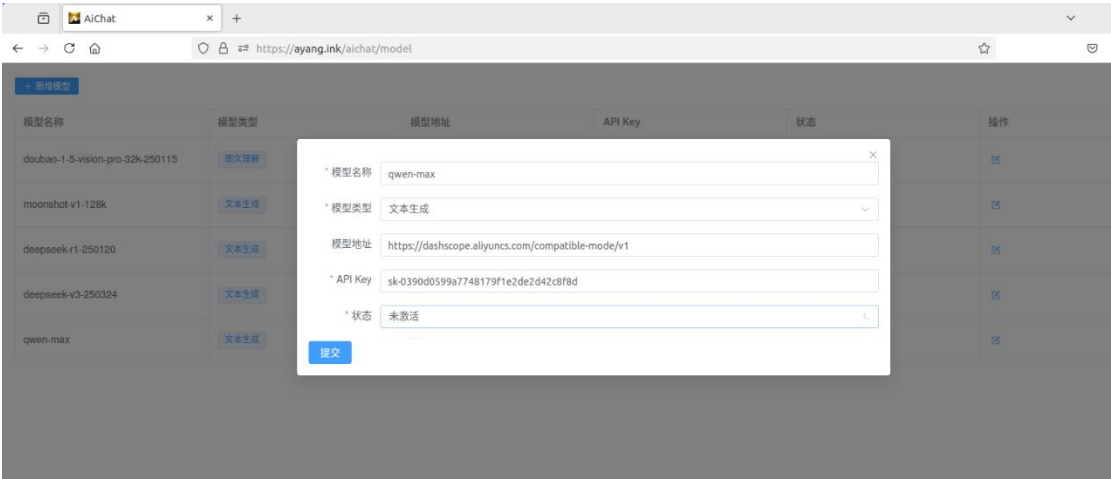


图 5.18 模型编辑页面

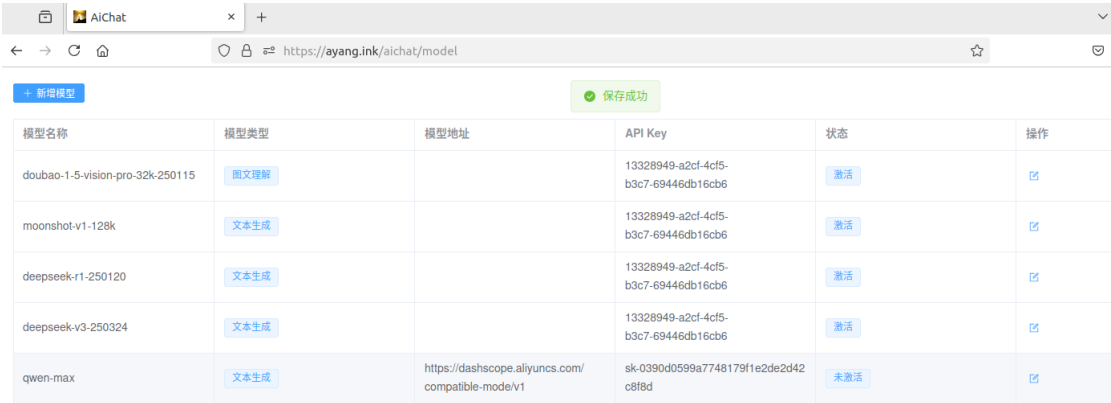


图 5.19 模型修改成功

如图 5.19 所示，修改成功会提示“保存成功”，模型信息会展示修改后的最新状态。

5.5 系统测试

5.5.1 系统测试环境

软件测试是软件开发中的不可或缺的一步，保证软件良好运行的重要基础^[19]。系统功能性测试主要是针对友为交流社区每一个模块重要的功能进行测试，测试的方式是通过不同的角度不同的方式来对具体的功能进行用例测试，以此来检验系统功能的实现是否符合我们的要求^[20]。系统测试环境分为软件环境和硬件环境，测试环境如表 5.1 所示：

表 5.1 系统测试环境

软件环境	
操作系统	Ubuntu 22.04
集成开发环境	Goland 2024
浏览器	FireFox
数据库	MySQL 5.7
数据库管理和设计工具	Navicat 15
接口测试工具	Postman
硬件环境	
CPU	AMD Ryzen7 5800h
RAM	32 G
核心数	16 C

5.5.2 系统功能测试

登录模块功能测试结果如表 5.2 所示：

表 5.2 登录模块测试记录

序号	功能名	测试用例	预期结果	结果
1	用户登录	点击 GitHub 跳转登录	登录成功，跳转首页	通过
2	管理员登录	输入正确账户密码	登录成功，跳转首页	通过
3	管理员登录	输入错误账户密码	登录失败，提示错误	通过
4	退出登录	点击“退出登录”	退出成功，跳转登录页面	通过
5	登录状态保持	关闭浏览器，重新进入首页	登录状态保持，无需重复登录	通过

会话模块功能测试结果如表 5.3 所示：

表 5.3 会话模块测试记录

序号	功能名	测试用例	预期结果	结果
1	创建、删除会话	点击“创建会话”和“删除会话”	创建、删除会话成功	通过
2	会话修改名称	点击修改会话名称，输入新名称，Enter	名称修改成功，显示新名称	通过
3	修改会话配置	修改会话配置，点击“保存配置”	显示“修改成功”	通过
4	文本对话	输入框输入问题，点击“发送”	能够正确回答问题	通过
5	图文理解对话	输入框添加图片输入问题，点击“发送”	能够正确理解问题，回答问题	通过
6	消息记录	多轮对话后刷新页面，点击会话	显示会话过往聊天记录	通过

知识库模块功能测试结果如表 5.4 所示：

表 5.4 知识库模块测试记录

序号	功能名	测试用例	预期结果	结果
1	知识库页面	点击“配置知识库”	进入知识库页面，展示知识库列表	通过
2	创建、修改和删除知识库	点击“创建知识库”“修改知识库”按钮，输入或修改信息，点击“确认”	提示“创建成功”或“修改成功”，知识库列表新增或显示知识库信息	通过
4	上传、下载文件	点击上传或下载文件	提示“上传成功”或“下载成功”	通过

模型配置模块功能测试结果如表 5.5 所示：

表 5.5 模型配置模块测试记录

序号	功能名	测试用例	预期结果	结果
1	模型配置页面	点击“配置模型”	进入模型配置页面， 展示模型信息列表	通过
2	新增、修改模型	点击"新增模型""修改知识库"按钮，输入或修改信息，点击“确认”	提示“创建成功”或“修改成功”，模型列表新增或显示模型信息	通过

5.6 本章小结

本章详细介绍了智能对话系统的各个核心模块的实现过程，包括登录、会话、知识库与模型配置模块，并通过图示展示了具体的界面与交互流程。最后，构建了完整的系统测试环境，对各模块的关键功能进行了功能性测试，验证了系统在实际使用中的稳定性与可靠性。

第六章 总结和展望

6.1 总结

本论文聚焦于“智能对话系统”的设计以及实现，探讨了在人工智能快速发展的大背景之下，怎样结合自然语言处理以及知识库问答等关键技术，构建出一个拥有实际应用能力的对话平台，智能对话系统作为人机交互的关键方式，已经渐渐渗透到客户服务、知识问答、教育辅导、企业协作等多个领域，成为推动人类社会信息化的关键工具。本研究选题紧跟技术发展前沿，有较强的理论研究价值与现实应用意义。

在系统设计以及实现的过程当中，本文构建了一个把会话管理、知识库构建与模型配置集成在一起的智能对话系统，与同类系统相比较，本系统有以下几个方面的优势：

1. 模块化设计且易于扩展，系统结构清晰，前后端相互分离，模块之间解耦，方便后期进行功能扩展以及技术升级。比如可以灵活替换模型后端，快速接入新的 API 服务。
2. 集成知识库问答能力，提高专业性，依靠引入基于向量检索的知识库模块，系统可结合用户自定义的文档资料，实现基于语义的智能问答，提升系统在垂直领域的回答准确率。
3. 用户体验得到优化，界面交互友好，系统前端采用现代化设计，支持 GitHub 三方登录，提供简洁的操作逻辑和清晰的提示反馈，整体交互流程流畅，提升了用户的使用体验。

6.2 展望

虽然本系统已经有基本的对话能力和知识问答能力，但是在开发过程中仍然存在一些不足之处，未来以及较大的优化空间，具体如下：

一是图像及多模态理解能力尚不全面，目前系统仅集成了基本的图文问答能力，对于更复杂的多模态任务如语音识别、视频内容理解、音视频语义融合等尚

未支持，限制了系统在教育、媒体、安防等领域的应用。

二是系统响应还有优化空间，目前数据都取自数据库。

针对以上不足，本文对未来系统的发展提出以下展望：

一是拓展多模态能力，未来应重点补齐音视频理解方面的能力，包括语音识别、语音合成、视频内容分析、音视频问答等功能，提升系统处理复杂场景信息的能力。

二是增加多级缓存，例如增加 Redis 组件作为二级缓存，增加进程缓存作为一级缓存，提高读场景的响应速度。

参考文献

- [1] XUE X, YU X, WANG F. ChatGPT chats on computational experiments: from interactive intelligence to imaginative intelligence for design of artificial societies and optimization of foundational models [J]. IEEE/CAA journal of automatica sinica,2023,10(6): 1357-1360.
- [2] 周纲,朱雯晶,张磊.以大模型重构下一代图书馆服务平台[J].信息与管理研究,2024,9(06):14-23.
- [3] APlayBoy. SELF-INSTRUCT: Aligning Language Models with Self-Generated Instructions 论文解读. <https://zhuanlan.zhihu.com/p/689082580>,2024
- [4] 张敖玮,王粟,宫士君.全面赋能体育科研: Open AI “未来已来”系列智能化工具应用研究[J].文体用品与科技.2024(24):196-198.
- [5] 孔月昕,邓攀.对话李开复:大模型公司的灵魂考验是什么[J].中国企业家.2024(08):26-31.
- [6] 白静,李哲.优化瀑布模型的网络课程开发模式研究[J].中国电化教育,2012,(05):94-99.
- [7] J. Meyerson. The Go Programming Language[J]. IEEE Software, 2014, 31(5): 104.
- [8] 曲锦旭.前后端分离模式在 Java 开发中的应用研究[J].信息与电脑(理论版),2024,36(08):19-21.J. Meyerson. The Go Programming Language[J]. IEEE Software, 2014, 31(5): 104.
- [9] 胡骏.基于 WEB 的制造业工艺管理系统设计与实现[D].大连理工大学,2016.
- [10] Dang Tran Khanh,Huy Ta Manh,Dang Ly Hoang,Le Hoang Nguyen. An Elastic Data Conversion Framework: A Case Study for MySQL and MongoDB[J]. SN Computer Science,2021,2(4).
- [11] Chuang H Y , Chen J L , Ma Y W .Malware Detection and Classification Based on Graph Convolutional Networks and Function Call Graphs[J].IT professional,2023,25(3):43-53.
- [12] Kazuo Matsumura,Hiroyuki Mizutani,Masahiko Arai. An Application of Structural Modeling to Software Requirements Analysis and Design.[J]. IEEE Trans. Software Eng.,1987,13(4).
- [13] 毕海天.基于前景理论的银行软件非功能需求评价研究[D].天津大学,2019.
- [14] 杨晓飞.基于移动 IM 的通信模块和网络增值服务模块的设计与实现[D].北京邮电大学,2015.
- [15] 王志涛.基于 B/S 模式的项目管理信息系统开发与设计[J].办公自动化,2024,29(24):84-86.
- [16] 李东旭.大型网络游戏高性能系统架构和并发技术研究[D].北京理工大学,2015.
- [17] 陈琳,赖秋玲,罗建国.面向全量用户的电力综合业务自助办理终端设计[J].电子测试,2019(10):80-82.
- [18] 熊良玉.荆州网通进销存设备平台设计与实现[D].大连理工大学,2018.
- [19] 李敬奇.单元测试中环境交互模型的设计与实现[D].北京邮电大学,2019.
- [20] 李晨.基于 Spring Boot 的电子商城设计与实现[D].哈尔滨工业大学,2020.

致谢

时光飞逝，转眼间四年紧张而又充实的大学生活即将画上句号。在这四年的学习生活中，我得到了许多来自老师、同学和朋友的关怀和帮忙。在学位论文即将完成之际，我要向所有在大学期间给予我支持、帮忙和鼓励的人献上我最诚挚的谢意。

首先，我要感谢我的指导老师蔡奕侨老师对我的教导。从毕设课题的选择，开题报告，到论文的撰写，蔡老师都给了我悉心的指导和热情的帮忙。蔡老师对工作的认真负责、对学术的钻研精神和严谨的学风，都是我终生学习的。不积跬步何以至千里，本设计能够顺利的完成，也归功于各位任课老师的认真负责，使我能够很好的掌握和运用专业知识，并在设计中得以体现。在此向华侨大学，计算机学院的全体老师表示由衷的谢意。感谢他们四年来的辛勤栽培最后，感谢我的家人对我的关爱和鼓励，以及所有陪我一路走来的同学和朋友，正是由于他们的支持和照顾，我才能安心学习，并顺利完成我的学业。

黄楊

2025 年 5 月 1 号